Deep Reinforcement Learning

Policy Gradients

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

PADERBORN UNIVERSITY

SciML

Content

Content

- From value-based methods to direct policy optimization
- Policy gradients
  - The objective
  - Evaluation the objective
  - Differentiating the objective
- The REINFORCE algorithm
- Understanding and challenges
- Reducing the variance
  - Baselines
  - Reward to go
- Off-policy policy gradients

- Some implementation details

Where are we?

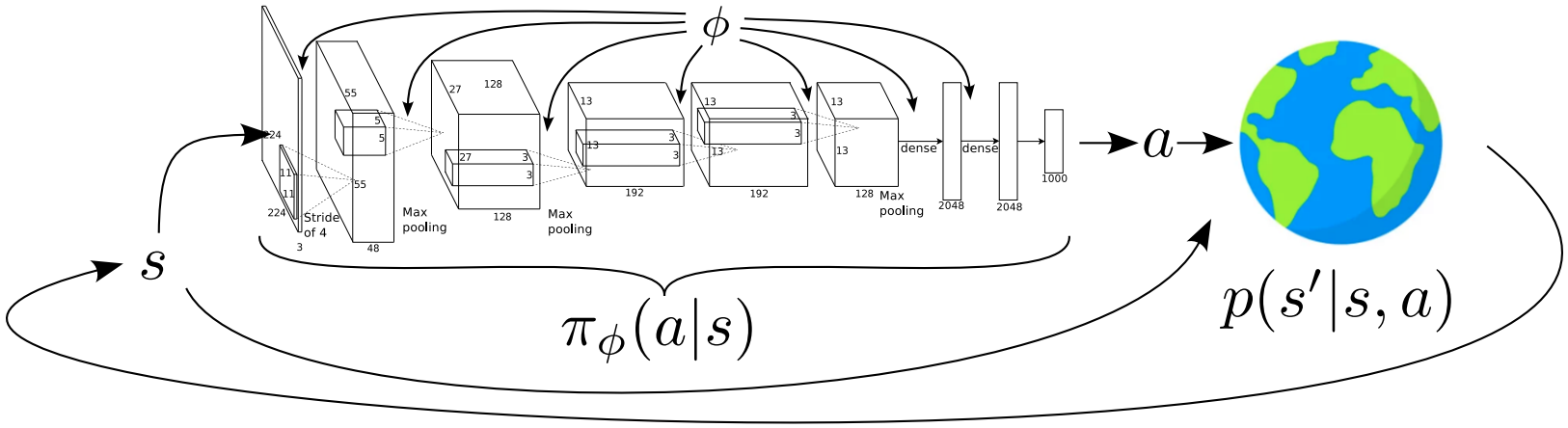
Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	
11	Advanced algorithms (Part I): From policy gradient to PPO	
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	
13	Exploration	
	Model-Based Control	
	Advanced Topics	

Lecture contents

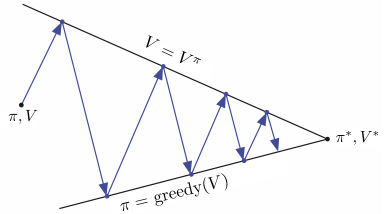
From value-based methods to direct policy optimization

From value-based methods to direct policy optimization

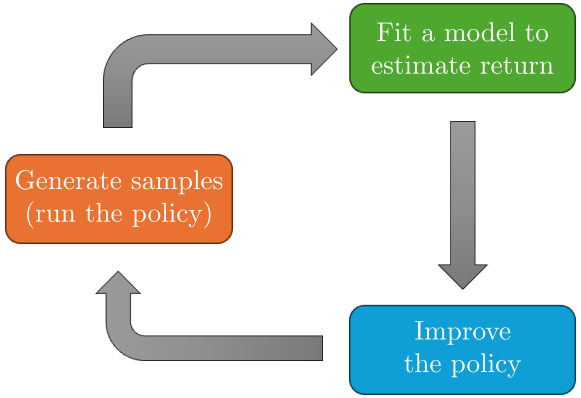
- Until now, all approaches followed the GPI concept.
- These are all **value-based methods**: Need an estimate of V^π or Q^π to improve π.
- But: what are we after in RL, really? $\Rightarrow$ The **optimal policy** π^*!



Inspired by Sergey Levine's CS285 lecture.



GPI (R. S. Sutton and Barto 1998).



Inspired by Sergey Levine's CS285 lecture.

Alternative approach:

1. Define probabilities over entire trajectories.

$$\Rightarrow \quad p_\phi(\underbrace{s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T}_{=\tau}) = p_\phi(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\phi(a_t|s_t) p(s_{t+1}|s_t, a_t).$$

2. Formulate RL objective; optimize dynamics via policy parameters ϕ.

$$\Rightarrow \quad \phi^* = \arg\max_\phi \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] = \arg\max_\phi \mathbb{E}_{s_0 \sim p(s_0)} \left[V^{\pi_\phi}(s_0) \right].$$

Policy gradients

The reinforcement learning objective

$$\phi^* = \arg\max_\phi \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] = \arg\max_\phi \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[r(\tau) \right]$$

Infinite horizon case: $\phi^* = \arg\max_\phi \mathbb{E}_{(s,a) \sim p_\phi(s,a)} \left[r \right]$

Finite horizon case: $\phi^* = \arg\max_\phi \sum_{t=0}^{T-1} \mathbb{E}_{(s_t,a_t) \sim p_\phi(s_t,a_t)} \left[r_t \right]$

- Infinite horizon: expected reward according to the stationary distributions of s and a.
- Finite horizon: linearity $\Rightarrow$ expected reward of individual stages (may differ with t).
- We are sticking to the finite-horizon case here.

💡 We are not considering the discount factor γ for now.

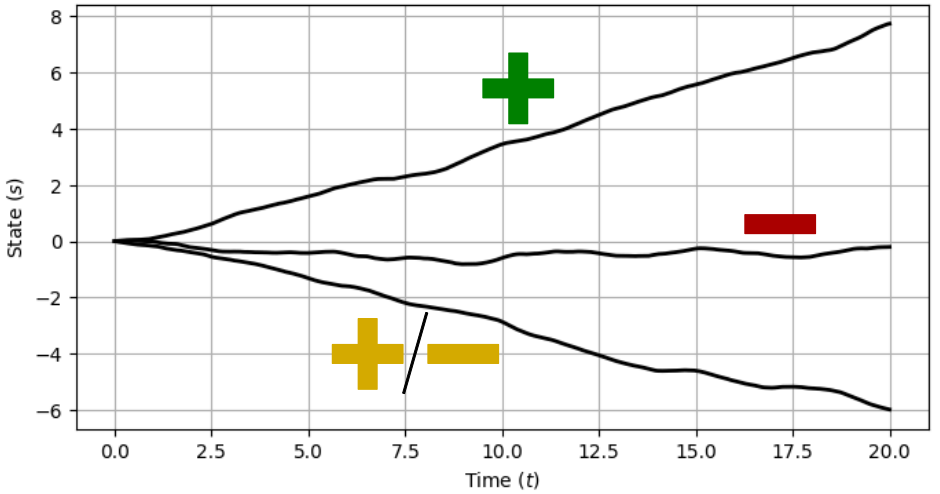
Evaluating the objective

$$\phi^* = \arg\max_{\phi} \underbrace{\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right]}_{L_\pi(\phi)}$$

- First, let's think about evaluating $L_\pi(\phi)$ for a fixed policy π.
- How do we estimate expectations if we don't have access to a model?

$\Rightarrow$ Monte Carlo sampling!

$$L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} r_{i,t}$$



Inspired by Sergey Levine's CS285 lecture.

- Sample N trajectories with s_0 according to the initial distribution and then following π.
- Take the sample average over these trajectories.

Policy gradient: Depending on the return, the plan is to increase or reduce the trajectories' likelihoods by adapting the policy π.

💡 The following **policy gradient theorem** was originally introduced in (Richard S. Sutton et al. 1999).

Differentiating the policy (1)

Definition of the expectation in continuous spaces

The expected value of a function $x(\tau)$ is

$$\mathbb{E}_{\tau\sim p}\left[x(\tau)\right] = \int p(\tau)x(\tau)\,\mathrm{d}\tau.$$

Here, p is the density according to which τ is distributed, with $\int p(\tau)\,\mathrm{d}\tau = 1$.

$$\phi^* = \arg\max_\phi \underbrace{\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\sum_{t=0}^{T-1} r_t\right]}_{L_\pi(\phi)}$$

$$L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}[\underbrace{r(\tau)}_{=\sum_{t=0}^{T-1} r_t}] = \int p_\phi(\tau)r(\tau)\,\mathrm{d}\tau$$

$$\begin{aligned}\nabla_\phi L_\pi(\phi) &= \nabla_\phi\left[\int p_\phi(\tau)r(\tau)\,\mathrm{d}\tau\right]\\ &= \int \nabla_\phi p_\phi(\tau) r(\tau)\,\mathrm{d}\tau\\ &= \int p_\phi(\tau)\nabla_\phi \log p_\phi(\tau) r(\tau)\,\mathrm{d}\tau\\ &= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right] \qquad (2)\end{aligned}$$

Note: Integration and differentiation are *linear* $\Rightarrow$ We can swap!

A convenient identity

$$\nabla_\phi p_\phi(\tau) = p_\phi(\tau)\frac{\nabla_\phi p_\phi(\tau)}{p_\phi(\tau)} = p_\phi(\tau)\nabla_\phi \log p_\phi(\tau) \qquad (1)$$

Differentiating the policy (2)

$$\phi^* = \arg\max_{\phi} L_\pi(\phi)$$

$$L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right] = \int p_\phi(\tau) r(\tau)\,\mathrm{d}\tau$$

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right]$$

$$\underbrace{p_\phi(s_0, a_0, \ldots, s_{T-1}, a_{T-1}, s_T)}_{=p_\phi(\tau)} = p(s_0)\prod_{t=0}^{T-1}\pi_\phi(a_t|s_t)p\left(s_{t+1}|s_t,a_t\right)$$

take log on both sides! ⇓ $(\log(a\cdot b) = \log(a) + \log(b))$

$$\log p_\phi(\tau) = \log p(s_0) + \sum_{t=0}^{T-1}\left[\log \pi_\phi(a_t|s_t) + \log p\left(s_{t+1}|s_t,a_t\right)\right]$$

$$\begin{aligned}
\nabla_\phi \log p_\phi(\tau) &= \nabla_\phi\left[\log p(s_0) + \sum_{t=0}^{T-1}\left[\log \pi_\phi(a_t|s_t) + \log p\left(s_{t+1}|s_t,a_t\right)\right]\right] \\
&= \nabla_\phi \log p(s_0) + \sum_{t=0}^{T-1}\left[\nabla_\phi \log \pi_\phi(a_t|s_t) + \nabla_\phi \log p\left(s_{t+1}|s_t,a_t\right)\right] \\
&= \underbrace{\nabla_\phi \log p(s_0)}_{=0} + \sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t) + \underbrace{\nabla_\phi \log p(s_{t+1}|s_t,a_t)}_{=0}
\end{aligned}$$

Policy gradient theorem (simplified)

$$\begin{aligned}
\nabla_\phi L_\pi(\phi) &= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\right) r(\tau)\right] \\
&= \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} r_t\right)\right]
\end{aligned}$$

Policy gradient theorem

What does this mean?

1. Goal: **find a policy that maximizes the value**,

$$\max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} \left[\sum_{t=0}^{T-1} r_t \right] = \max_{\pi} \mathbb{E}_{\tau \sim p^{\pi}} \left[r(\tau) \right].$$

Policy gradient theorem (simplified)

$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right]$$

2. **Neural network approximator** $\pi_{\phi} \Rightarrow$ maximization w.r.t. the policy parameters:

$$\max_{\phi} \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[r(\tau) \right] = \max_{\phi} L_{\pi}(\phi).$$

3. **Challenging objective function**: Integration over τ (i.e., the space of trajectories) is infeasible!

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[r(\tau) \right] = \int p_{\phi}(\tau) r(\tau) \, \mathrm{d}\tau.$$

4. Use $\log$ identity to derive a formulation of the gradient that we can *approximate using sampling*:

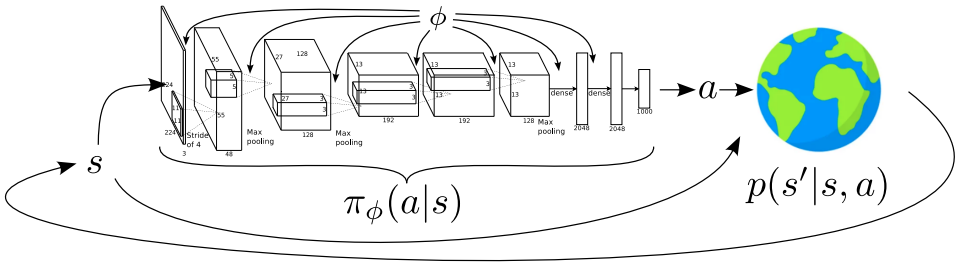
$$\nabla_{\phi} L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi} \log \pi_{\phi}(a_t | s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \right].$$

REINFORCE

We now have a formulation of the policy gradient that we can **approximate using Monte Carlo sampling**:

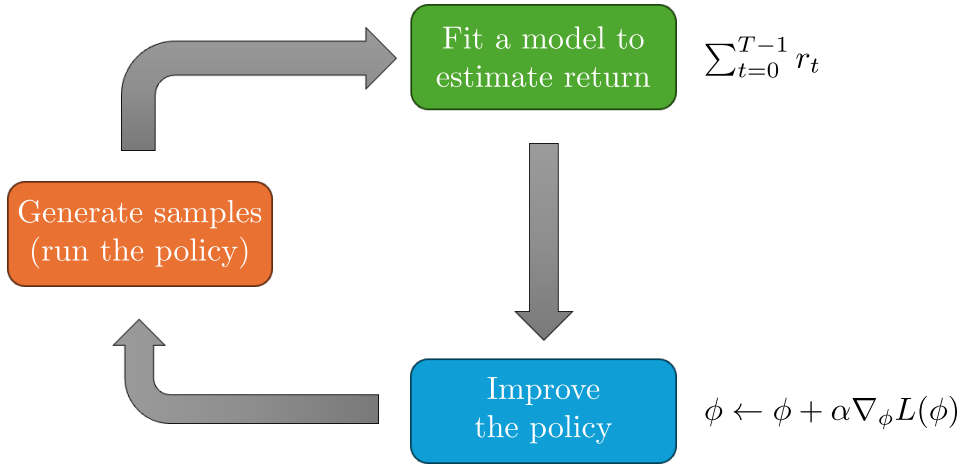
$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_t|s_t)\right)\left(\sum_{t=0}^{T-1} r_t\right)\right] \approx \frac{1}{N}\sum_{i=1}^{N}\underbrace{\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\right)}_{\text{(I)}}\underbrace{\left(\sum_{t=0}^{T-1} r_{i,t}\right)}_{\text{(II)}}.$$

(I) This term is simpliy the derivative of the policy network w.r.t. the weights $\Rightarrow$ backpropagation!
(II) This term is the experience we collect from interactions with the environment.



The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow$ $\{((s_{i,0},a_{i,0},r_{i,0}),\ldots,(s_{i,T-1},a_{i,T-1},r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L_\pi(\phi)$ $\approx \frac{1}{N}\sum_{i=1}^N\left(\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\right)\left(\sum_{t=0}^{T-1} r_{i,t}\right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha\nabla_\phi L_\pi(\phi)$.



Understanding the policy gradient

Continuous action example: A Gaussian policy

- Assume a **Gaussian policy** whose mean is given by a neural network:

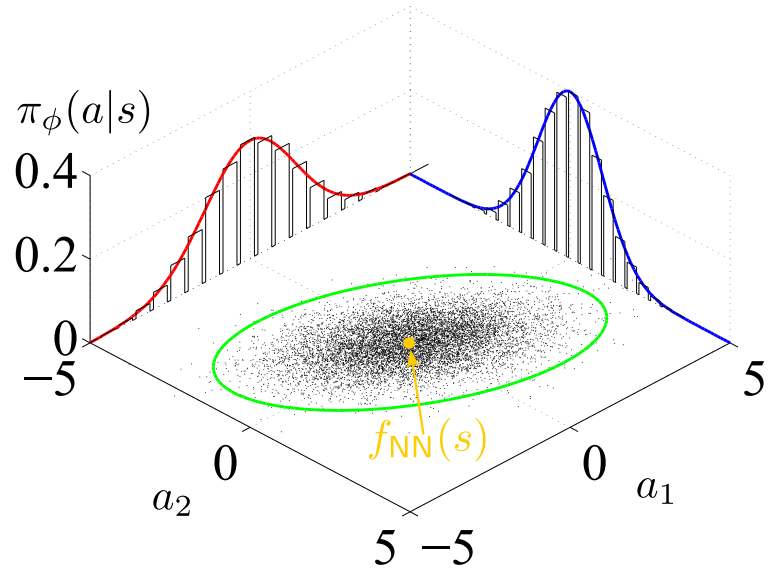
$$\begin{aligned}\pi_\phi(a|s) &= \mathcal{N}\left(f_{\text{NN}}(s), \Sigma\right) \\ &= \frac{1}{C}\exp\left(-\frac{1}{2}(a - f_{\text{NN}}(s))^\top \Sigma^{-1}(a - f_{\text{NN}}(s))\right) \\ &= \frac{1}{C}\exp\left(-\frac{1}{2}\|a - f_{\text{NN}}(s)\|_\Sigma^2\right).\end{aligned}$$

- Taking the log (with $\log C^{-1} = -\log C$) yields:

$$\log \pi_\phi(a|s) = -\frac{1}{2}\|a - f_{\text{NN}}(s)\|_\Sigma^2 - \log C.$$

- Taking the gradient leads to the following expression:

$$-\frac{1}{2}\Sigma^{-1}(f_{\text{NN}}(s) - a)\frac{\partial f_{\text{NN}}}{\partial \phi}.$$



Adapted from Wikipedia.

The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow$ $\{((s_{i,0}, a_{i,0}, r_{i,0}), \ldots, (s_{i,T-1}, a_{i,T-1}, r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L_\pi(\phi)$ $\approx \frac{1}{N}\sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\right)\left(\sum_{t=0}^{T-1} r_{i,t}\right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.

Comparison to maximum likelihood

- Consider the task of maximizing the likelihood L of a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ in supervised learning, e.g., using a neural network with parameter vector θ:

$$\max_\theta L(\theta) = \max_\theta p_\theta(y_1|x_1) \times \ldots \times p_\theta(y_N|x_N)$$

$$= \max_\theta \prod_{i=1}^N p_\theta(y_i|x_i).$$

- Optimizing over products is hard $\Rightarrow$ take the log:

$$\log L(\theta) = \log \left(\prod_{i=1}^N p_\theta(y_i|x_i) \right) = \sum_{i=1}^N \log p_\theta(y_i|x_i).$$

Maximum likelihood gradient:

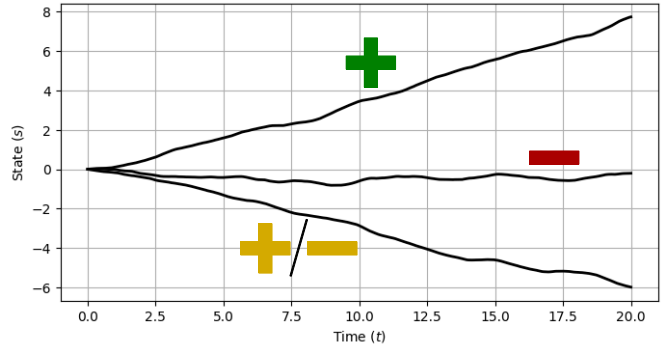
$$\nabla_\theta \log L(\theta) = \sum_{i=1}^N \nabla_\theta \log p_\theta(y_i|x_i).$$

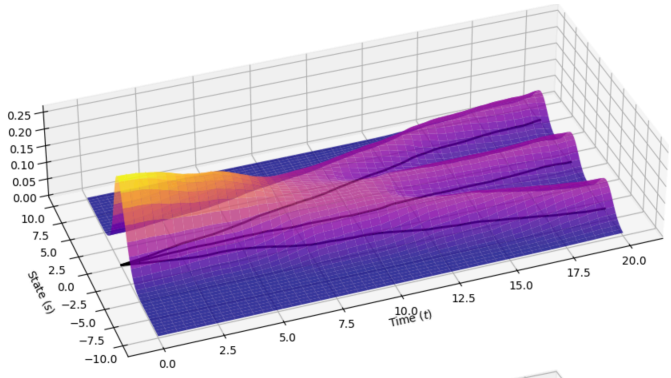
Comparison against the policy gradient (MC sampling):

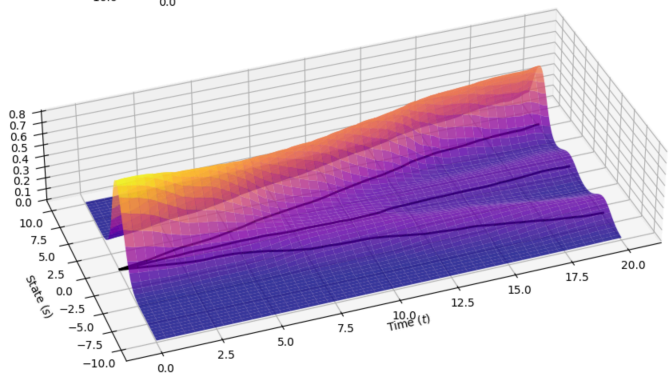
$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t=0}^{T-1} r_{i,t} \right).$$

- The policy gradient can be seen as a weighted version of the maximum likelihood objective.
- Why is weighting is necessary?
  - In maximum likelihood, we always want to maximize the likelihood of the “correct” labels.
  - In policy gradients, we may also want to reduce the likelihood of low-reward samples!

Understanding the policy gradient







Inspired by Sergey Levine's CS285 lecture.

- What did we just do?
  $\Rightarrow$ let's rewrite the formula a little and *compare against maixmum likelihood*:

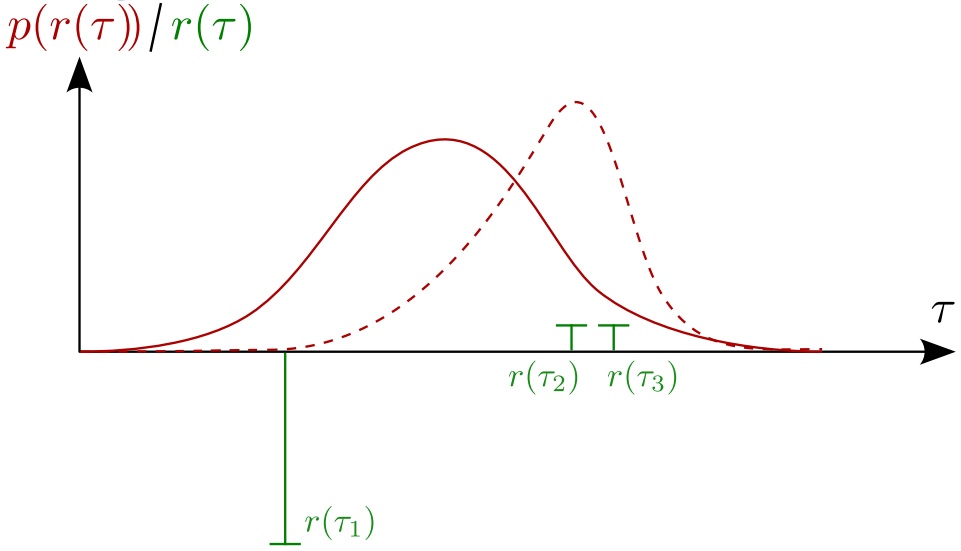
$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \underbrace{\nabla_\phi \log \pi_\phi(\tau_i)}_{\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})} \underbrace{r(\tau_i)}_{\sum_{t=0}^{T-1} r_{i,t}} \quad \text{vs.} \quad \nabla_\theta L(\theta) = \sum_{i=1}^{N} \nabla_\theta \log \pi_\phi(\tau_i).$$

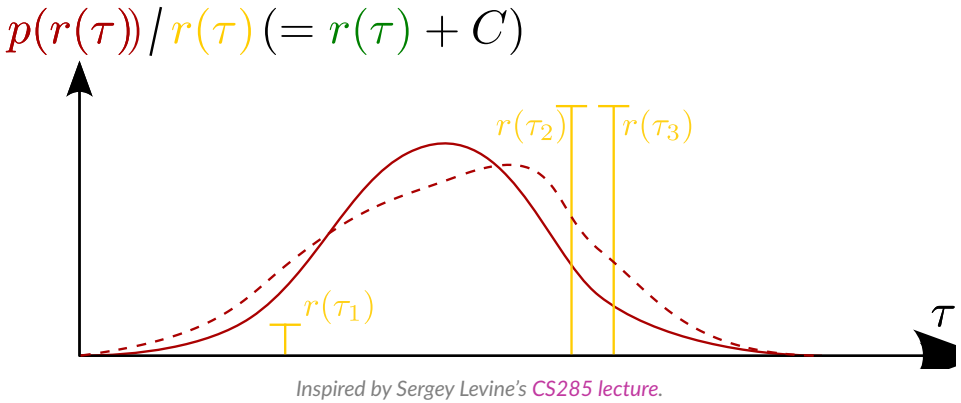
- Good experience is made more likely: We increase the proability of the policy to produce similar trajectories.
- Bad experience is made less likely.
- This simply formalizes the notion of "trial and error"!

A note on partial observability (without going into details)
The policy gradient also holds for partially observed MDPs (POMDPs). That is, for policies $\pi(a|o)$. In simple terms, the reason is that the policy gradient theorem does not make use of the Markov property.

Challenges with policy gradients

High variance





Inspired by Sergey Levine's CS285 lecture.

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) r(\tau_i).$$

- Adding a constant to the reward should not change the optimal policy!
  - This is true for *any* optimization problem, e.g.,

$$\arg\min_x f(x) = \arg\min_x [f(x) + 1000].$$

- In the limit $N \to \infty$, this is true for policy gradients as well…
  - … but it does not hold for finite sample sizes, in particular few sample trajectories.
- Depending on the sign of $r(\tau)$, we **either increase or decrease** the probability of seeing similar trajectories τ_i (and their rewards $r(\tau_i)$) in the future.
- This is an instance of the **high variance** issue with policy gradients.
- An even more "catastropic" version of this: What if we scale some of the rewards to be exactly zero?

Reducing variance – baselines

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) r(\tau_i).$$

- **Question**: Is there a systematic way to fix the challenge of reward offsets?
  $\Rightarrow$ Let's balance the "weight" of of our likelihood objective in such a way that …
  - Better-than-average rewards *increase* the probability of the respective trajectories.
  - Worse-than-average rewards *decrease* the probability.
- Several **advantages**!
  - This aligns with our intuition that better-than-average is reinforced and worse-than-average is penalized.
  - This approach makes the policy gradient independent of the actual size of the reward signal.

Approach: subtract a **baseline** b from the reward signal

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_\phi \log \pi_\phi(\tau_i) [r(\tau_i) - b],$$

where $b = \frac{1}{N} \sum_{i=1}^{N} r(\tau_i)$.

Theorem
Subtracting any constant b is *unbiased in expectation*

Proof: Start with the exact formulation of the policy gradient following (2),

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\nabla_\phi \log p_\phi(\tau) (r(\tau) - b) \right].$$

Unbiased in expectation $\Rightarrow$ baseline term is zero in expectation:

$$\mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\nabla_\phi \log p_\phi(\tau) b \right] = \int p_\phi(\tau) \nabla_\phi \log p_\phi(\tau) b \, \mathrm{d}\tau$$

$$\overset{(1)}{=} \int \nabla_\phi p_\phi(\tau) b \, \mathrm{d}\tau = b \nabla_\phi \int p_\phi(\tau) \, \mathrm{d}\tau = (\nabla_\phi 1) b = 0. \qquad \square$$

The optimal baseline

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau)(r(\tau)-b)\right].$$

How do we find the optimal baseline?
$\Rightarrow$ optimization: minimize the variance

$$\text{var}\left[x\right] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2$$

$$\text{var} = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\underbrace{(\nabla_\phi \log p_\phi(\tau)}_{=g(\tau)}(r(\tau)-b))^2\right] - \left(\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau)\left(r(\tau)-\cancel{b}\right)\right]\right)^2 \quad \text{(unbiased baseline)}$$

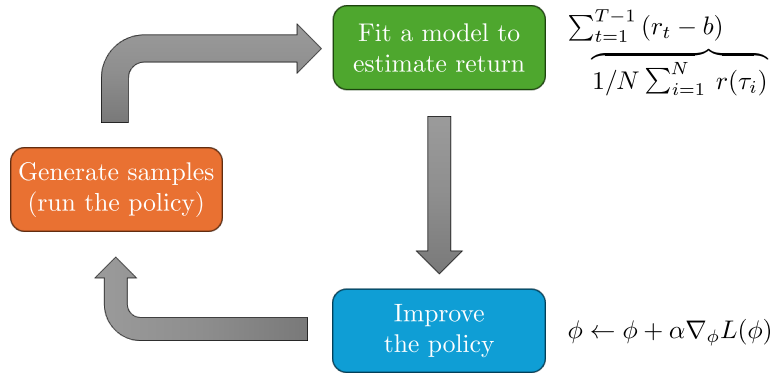
$$\begin{aligned}
\frac{\mathrm{dvar}}{\mathrm{d}b} &= \frac{\mathrm{d}}{\mathrm{d}b}\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2(r(\tau)-b)^2\right] = \frac{\mathrm{d}}{\mathrm{d}b}\left(\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau)^2\right] - 2\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau) b\right] + \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 b^2\right]\right) \\
&= \frac{\mathrm{d}}{\mathrm{d}b}\left(\cancel{\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau)^2\right]} - 2b\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau)\right] + b^2\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2\right]\right) \\
&= -2\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau)\right] + 2b\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2\right] \stackrel{!}{=} 0.
\end{aligned}$$

The optimal baseline b^* is the expected reward, *weighted by gradient magnitudes*:

$$b^* = \frac{\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2 r(\tau)\right]}{\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[g(\tau)^2\right]}.$$

Note: b^* is hard to calculate. In practice: usually take average reward

$$b = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right] \approx \frac{1}{N}\sum_{i=1}^{N} r(\tau_i).$$



Inspired by Sergey Levine's CS285 lecture.

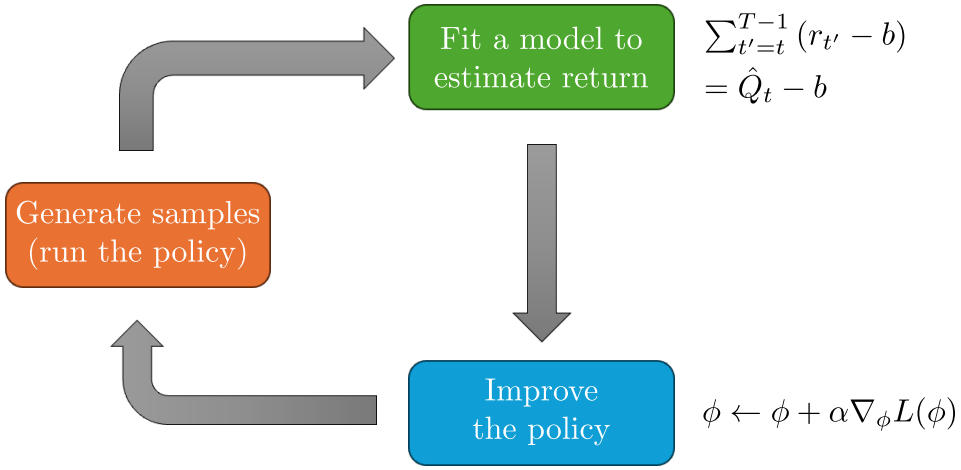
Reducing variance – causality

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \right) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

- **Causality**: Policy at time t' cannot affect reward at time t when $t < t'$.
  - "What you do now, is not going to change the rewards you received in the past."
- **Question**: Are we making use of causality in the above equation?
  - Let's rewrite it and make use of the distributive property ($a \cdot (b + c) = a \cdot b + a \cdot c$):

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \left(\sum_{t'=0}^{T-1} r_{i,t'} \right).$$

    $\Rightarrow$ Past rewards (i.e., $t' < t$) have an impact on the policy π_ϕ!
  - In expectation, these factors have to cancel out (and one can prove this). **But**: for finite sample sizes, they do not and instead increase the variance.
- **Simple fix**: "*reward to go*" $\hat{Q}_{i,t} = \sum_{t'=t}^{T-1} r_{i,t'}$ (the only change is $0 \rightarrow t$),



Inspired by Sergey Levine's CS285 lecture.

💡 Do not confuse causality with the Markov property! The Markov propery (which may hold for a system, but does not have to) says that your future states do not depend on past states, just the present state. Causality is always true: "Rewards in the past are independent of decisions in the present."

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \hat{Q}_{i,t}.$$

Off-policy policy gradients

Why policy gradients are on-policy

- Recall the policy gradient optimization problem:

$$\phi^* = \arg\max_\phi L_\pi(\phi) = \arg\max_\phi \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right].$$

- The corresponding policy gradient (for simplicity, without baseline) is

$$\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[\nabla_\phi \log p_\phi(\tau) r(\tau)\right].$$

- The problem: $\mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right]$ requires on-policy sampling!
- We cannot skip step 1 of the REINFORCE algorithm.

The REINFORCE algorithm

1. Sample $\{\tau_i\}_{i=1}^N$ using $\pi_\phi(a|s) \Rightarrow$ $\{((s_{i,0},a_{i,0},r_{i,0}),\ldots,(s_{i,T-1},a_{i,T-1},r_{i,T-1}))\}_{i=1}^N$.
2. $\nabla_\phi L_\pi(\phi) \approx \frac{1}{N}\sum_{i=1}^N \left(\sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})\right)\left(\sum_{t=0}^{T-1} r_{i,t}\right)$.
3. Gradient ascent: $\phi \leftarrow \phi + \alpha\nabla_\phi L_\pi(\phi)$.

The trouble in Deep RL:

- Neural networks usually require small update steps ...
- ... but a large number of steps!
- On-policy learning can become highly inefficient!

Importance sampling (continuous setting)

Importance sampling
How do we calculate the expectation *w.r.t. a different distribution*?

$$\begin{aligned}\mathbb{E}_{x\sim p}\left[f(x)\right] &= \int p(x) f(x)\,\mathrm{d}x \\ &= \int \frac{q(x)}{q(x)} p(x) f(x)\,\mathrm{d}x \\ &= \int q(x) \frac{p(x)}{q(x)} f(x)\,\mathrm{d}x \\ &= \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right].\end{aligned}$$

💡 This formula is exact!

- Back to the reinforcement learning objective

$$L_\pi(\phi) = \mathbb{E}_{\tau\sim p_\phi(\tau)}\left[r(\tau)\right].$$

- What if we have trajectory samples from $\overline{p}$ instead of p_ϕ?
  - Examples are previous policies, expert demonstrations, combinations thereof, …
- Simply transfer of the importance sampling formula:

$$L_\pi(\phi) = \mathbb{E}_{\tau\sim\overline{p}(\tau)}\left[\frac{p_\phi(\tau)}{\overline{p}(\tau)} r(\tau)\right].$$

- Insert definition of trajectory probabilities:

$$p_\phi(\tau) = p(s_0)\prod_{t=0}^{T-1}\pi_\phi(a_t|s_t)p\left(s_{t+1}|s_t,a_t\right),$$

$$\frac{p_\phi(\tau)}{\overline{p}(\tau)} = \frac{\cancel{p(s_0)}\prod_{t=0}^{T-1}\pi_\phi(a_t|s_t)\cancel{p\left(s_{t+1}|s_t,a_t\right)}}{\cancel{p(s_0)}\prod_{t=0}^{T-1}\overline{\pi}(a_t|s_t)\cancel{p\left(s_{t+1}|s_t,a_t\right)}} = \prod_{t=0}^{T-1}\frac{\pi_\phi(a_t|s_t)}{\overline{\pi}(a_t|s_t)}. \quad (3)$$

Off-policy policy gradient with importance sampling (1)

- Once more, recall the loss function $L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)}[r(\tau)]$.
- **Question**: Can we estimate L for ϕ' using samples according to p_ϕ?

$$L_\pi(\phi') = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[\frac{p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau)\right]$$

- Note that only $p_{\phi'}(\tau)$ actually depends on the new parameter vector ϕ':

$$\nabla_{\phi'} L_\pi(\phi') = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[\frac{\nabla_{\phi'} p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau)\right] = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[\frac{\nabla_{\phi'} p_{\phi'}(\tau)}{p_\phi(\tau)} r(\tau)\right] = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[\frac{p_{\phi'}(\tau)}{p_\phi(\tau)} \nabla_{\phi'} \log p_{\phi'}(\tau) r(\tau)\right].$$

- It's the policy gradient formula we know, modified by the importance sampling weights!
  - In fact, that's an alternative way to derive it: introduce importance sampling in the policy gradient formula $\nabla_\phi L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)}[\nabla_\phi \log p_\phi(\tau) r(\tau)]$.
- Setting $\theta' = \theta$, we recover the on-policy policy gradient.

A convenient identity

$$p_\phi(\tau) \nabla_\phi \log p_\phi(\tau) = \nabla_\phi p_\phi(\tau)$$

💡 As derived earlier, using $p_{\phi'}(\tau)$ or $\pi_{\phi'}(\tau)$ (i.e., the product over action probabilities along the trajectories) yields the same result

$$\nabla_\phi \log p_\phi(\tau) = \cancel{\nabla_\phi \log p(s_0)} + \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_t|s_t) + \cancel{\nabla_\phi \log p(s_{t+1}|s_t, a_t)} = \nabla_\phi \log \pi_\phi(\tau).$$

Off-policy policy gradient with importance sampling (2)

- Now that we have the formula, let's reformulate and introduce causality again:

$$\nabla_{\phi'} L_\pi(\phi') = \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\frac{p_{\phi'}(\tau)}{p_\phi(\tau)} \nabla_{\phi'} \log \pi_{\phi'}(\tau) r(\tau) \right]$$

$$= \mathbb{E}_{\tau \sim p_\phi(\tau)} \Big[\underbrace{\left(\prod_{t=0}^{T-1} \frac{\pi_{\phi'}(a_t|s_t)}{\pi_\phi(a_t|s_t)} \right)}_{\text{Eq. (3)}} \left(\sum_{t=0}^{T-1} \nabla_{\phi'} \log \pi_{\phi'}(a_t|s_t) \right) \left(\sum_{t=0}^{T-1} r_t \right) \Big]$$

(Causality: Future actions don't affect (1) the current weights and (2) past rewards. ⇒ Distribute weights!)

$$= \mathbb{E}_{\tau \sim p_\phi(\tau)} \left[\left(\sum_{t=0}^{T-1} \nabla_{\phi'} \log \pi_{\phi'}(a_t|s_t) \left(\prod_{t'=0}^{t} \frac{\pi_{\phi'}(a_{t'}|s_{t'})}{\pi_\phi(a_{t'}|s_{t'})} \right) \right) \left(\sum_{t=0}^{T-1} r_t \left(\prod_{t''=t}^{t'} \frac{\pi_{\phi'}(a_{t''}|s_{t''})}{\pi_\phi(a_{t''}|s_{t''})} \right) \right) \right]$$

- Ignoring the last term yields some form of policy iteration algorithm (more on this later):

$$\cancel{\left(\prod_{t''=t}^{t'} \frac{\pi_{\phi'}(a_{t''}|s_{t''})}{\pi_\phi(a_{t''}|s_{t''})} \right)}.$$

Some implementation details

Policy gradient with automatic differentiation

$$\nabla_\phi L_\pi(\phi) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T-1} \nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}) \underbrace{\hat{Q}_{i,t}}_{=\sum_{t'=t}^{T-1} r_{i,t'}} .$$

- Calculation of $\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t})$ can be very inefficient!
  - Consider a network with $d = 10^6$ weights and a dataset of $N = 100$ trajectories of length $T = 100$ each.
  - We would have to calculate a vector with 10^6 entries $NT = 10^5$ times!
- **Alternative**: Compute policy gradients using automatic differentiation (e.g., Python's pyTorch, TensorFlow, JAX).
  - We need a graph such that its gradient is the desired policy gradient.
  - Recall the maximum likelihood objective from supervised ML:

Pseudocode example

```
# Define a simple policy network (actor)
class PolicyNetwork(nn.Module):
    def __init__(self, state_dim, action_dim):
        super(PolicyNetwork, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(state_dim, 64),
            nn.ReLU(),
            nn.Linear(64, action_dim)
        )

    def forward(self, state):
        logits = self.fc(state)
        return Categorical(logits=logits) # Outputs a policy distribution

# Initialize network and optimizer
policy = PolicyNetwork(state_dim=n, action_dim=m)
optimizer = optim.Adam(policy.parameters(), lr=0.01)

# Forward pass: get the action distribution for the current state
dist = policy(state)

# Calculate the log probability of the action that was actually taken
log_prob = dist.log_prob(action_taken)

# Define the Pseudo-Loss: -log_prob * Reward ("-" due to gradient descent)
pseudo_loss = -(log_prob * reward)

# Autodiff calculates the policy gradient
```

$$\nabla_\phi J_{\mathrm{ML}}(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\nabla_\phi \log \pi_\phi(a_{i,t}|s_{i,t}), \quad \Rightarrow \quad J_{\mathrm{ML}}(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\log \pi_\phi(a_{i,t}|s_{i,t}).$$

```
pseudo_loss.backward()
optimizer.step()
```

- **Solution**: Just implement a *pseudo-loss* as a weighted maximum likelihood

$$\tilde{L}(\phi) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\log \pi_\phi(a_{i,t}|s_{i,t})\hat{Q}_{i,t}.$$

Policy gradient in practice

Some practical considerations / differences to supervised learning

- Remember that the **policy gradient has high variance**.
  - This isn't the same as supervised learning!
  - Gradients will be really noisy!
- Consider using **much larger batches**.
  - Several 1000s is quite common.
- Tweaking learning rates is very hard.
  - Adaptive step size rules like ADAM can be OK.
  - More details later.

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.

Sutton, Richard S, David McAllester, Satinder Singh, and Yishay Mansour. 1999. “Policy Gradient Methods for Reinforcement Learning with Function Approximation.” In *Advances in Neural Information Processing Systems*, edited by S. Solla, T. Leen, and K. Müller. Vol. 12. MIT Press.